

KCC ID API Guide (Central Authentication for the Ecosystem)

For: Ecosystem app developers building or integrating applications: PlenorHub intengration, Shuyi, and Kincore with centralized login and access tokens.

Version: 4

Updated: May 17, 2026

0) Terminology & Naming

- **Product name:** KCC ID
 - **Service / host shorthand:** kccid
 - **API namespace:** /kccid/...
 - **Production host:** <https://auth.bigkpay.com>
 - **Dashboard:** <https://id.kccdigital.com>
-

Base URLs

- KCC_ID_BASE_URL = <https://auth.bigkpay.com>
- KCC_ID_JWKS_URL = <https://auth.bigkpay.com/.well-known/jwks.json>
- KCC_ID_OPENID_CONFIG_URL = <https://auth.bigkpay.com/.well-known/openid-configuration>

Production check:

- GET /.well-known/openid-configuration is live on <https://auth.bigkpay.com>
 - GET /.well-known/jwks.json is live on <https://auth.bigkpay.com>
 - The discovery document advertises the versioned public auth endpoints under /kccid/v1/*
-

1) What KCC ID Is

KCC ID is the **central identity + authorization** service for the ecosystem.

It provides:

- **Identity** (KCC user IDs, passwords, profile basics)
- **OAuth-style token issuance** (Authorization Code + PKCE, Refresh Token)
- **JWTs** with a shared claim contract consumed by downstream services
- **JWKS/OpenID discovery** endpoints for token validation
- **OTP flows** (registration and password reset)
- **TOTP-based two-factor authentication** (Google Authenticator / Authy)

- **Internal authorization sync** API used by backend services
 - **Admin dashboard** for user management, audit logs, and security monitoring
-

2) Environments

Environment	Base URL
Production	https://auth.bigkpay.com

All endpoint paths in this guide are relative to the base URL.

3) Discovery Endpoints (Public)

3.1 JWKS (public keys)

- **GET** /.well-known/jwks.json

Used by services to validate RS256 JWT signatures.

3.2 OpenID configuration

- **GET** /.well-known/openid-configuration

Returns the issuer and endpoint URLs: `authorization_endpoint`, `token_endpoint`, `userinfo_endpoint`, `jwks_uri`, and supported grant/response types.

4) OAuth-Style Token Flows

KCC ID supports:

- **Authorization Code + PKCE** (recommended for web/mobile)
- **Refresh Token** rotation
- A **compatibility login** endpoint (direct username/password → tokens)

Key details:

- POST /kccid/v1/authorize is a **non-redirect credential-to-authorization-code** endpoint.
- POST /kccid/v1/login is a **direct credential-to-token compatibility endpoint**.
- New integrations must use the PKCE flow via /kccid/v1/authorize + /kccid/v1/token.
- Unversioned /kccid/* routes are deprecated.

4.1 Supported client IDs

- bigk_wallet
- bigk_admin
- bigk_merchant

- plenorhub_admin
- shuyi
- kincore

A request using an unsupported client_id returns 401 { "error": "invalid_client" }.

5) Endpoint Reference

5.1 Authorize (PKCE)

- **POST** /kccid/v1/authorize
- Rate limited: throttle:login (5/min per IP)

Request body:

```
{
  "identifier": "user@example.com",
  "password": "Secret123!",
  "client_id": "shuyi",
  "scope": "openid profile email",
  "code_challenge": "<pkce_code_challenge>",
  "code_challenge_method": "S256"
}
```

Response (200):

```
{
  "code": "<uuid>",
  "token_type": "authorization_code",
  "expires_in": 600
}
```

5.2 Token (authorization_code or refresh_token)

- **POST** /kccid/v1/token

5.2.1 Exchange authorization code

```
{
  "grant_type": "authorization_code",
  "code": "<uuid>",
  "client_id": "shuyi",
  "code_verifier": "<pkce_code_verifier>"
}
```

Standard response:

```
{
  "access_token": "<jwt>",
  "token_type": "Bearer",
  "id_token": "<jwt>",
}
```

```

    "refresh_token": "<uuid>",
    "expires_in": 3600,
    "scope": "openid profile email"
}

```

2FA challenge response (when TOTP is enabled):

```

{
  "requires_2fa": true,
  "challenge_token": "<jwt>",
  "message": "Two-factor authentication required."
}

```

When a 2FA challenge is returned, the client must call POST /kccid/v1/2fa/verify with the challenge token and a TOTP code to receive the final tokens.

5.2.2 Refresh token

```

{
  "grant_type": "refresh_token",
  "refresh_token": "<uuid>",
  "client_id": "shuyi"
}

```

Refresh tokens are **rotated** — the previous token is revoked after each use.

5.3 Login (compat: username/password → tokens)

- **POST** /kccid/v1/login
- Rate limited: throttle:login

```

{
  "identifier": "user@example.com",
  "password": "Secret123!",
  "client_id": "shuyi",
  "scope": "openid profile email"
}

```

Response: same shape as /kccid/v1/token success.

5.4 Userinfo (requires Bearer token)

- **GET** /kccid/v1/userinfo

```

{
  "sub": "<kcc_user_id>",
  "email": "user@example.com",
  "email_verified": true,
  "name": "Jane Doe",
  "preferred_username": "jane_doe",
  "role": "app",
  "token_type": "app",
  "wallet_id": 123
}

```

5.5 Sessions (requires Bearer token)

- **GET** /kccid/v1/sessions — list active sessions
- **POST** /kccid/v1/sessions/revoke — revoke a session by session_id
- **POST** /kccid/v1/logout — revoke the supplied refresh token

5.6 Connected Apps (requires Bearer token)

- **GET** /kccid/v1/connected-apps — list apps the user has authorized
-

6) Two-Factor Authentication (2FA / TOTP)

KCC ID supports TOTP-based 2FA using apps like Google Authenticator or Authy.

6.0 Which clients enforce 2FA?

2FA is currently **enforced only for bigk_wallet**. When a wallet user logs in (via /authorize or /login), the server checks if 2FA is required before issuing tokens. Other clients (bigk_admin, bigk_merchant, plenorhub_admin, shuyi, kincore) skip the 2FA check entirely and get tokens directly.

The enforcement list is controlled by KCC_ID_2FA_REQUIRED_CLIENTS (comma-separated, defaults to bigk_wallet).

2FA login flow (wallet):

1. User calls /kccid/v1/authorize with credentials
2. If TOTP is enabled: returns 2fa_required + challenge_token + methods: ["totp", "backup_code"]
3. If TOTP is NOT enabled but required: returns 2fa_enrollment_required + challenge_token (client should show the TOTP setup screen)
4. Client calls /kccid/v1/2fa/verify with the challenge token + code
5. Server issues full token set

Non-wallet clients: Steps 2-4 are skipped. Tokens are issued directly after credential validation.

6.1 Setup TOTP

- **POST** /kccid/v1/2fa/totp/setup
- Rate limited: throttle:otp
- Auth: Bearer token **or** challenge_token in request body

Generates a secret and provisioning URI. Accepts either a challenge_token from login or a Bearer token. The user resolves from whichever is provided.

Response:

```
{  
  "secret": "JBSWY3DPEHPK3PXP",
```

```
  "provisioning_uri":  
  "otpauth://totp/KCC%20ID:user@example.com?secret=JBSWY3DPEHPK3PXP&issuer=KCC%  
  20ID"  
}
```

6.2 Enable TOTP

- **POST** /kccid/v1/2fa/totp/enable
- Rate limited: throttle:otp
- Auth: Bearer token **or** challenge_token in request body

```
{  
  "secret": "JBSWY3DPEHPK3PXP",  
  "code": "123456"  
}
```

Response:

```
{  
  "message": "Two-factor authentication enabled.",  
  "backup_codes": ["a1b2c3d4", "e5f6g7h8", "i9j0k1l2", "m3n4o5p6",  
  "q7r8s9t0"]  
}
```

Important: Enabling TOTP increments token_version, which invalidates all existing sessions. The user must re-authenticate. Store the backup codes — they are only shown once.

6.3 Verify 2FA Challenge

- **POST** /kccid/v1/2fa/verify
- Rate limited: throttle:otp

```
{  
  "challenge_token": "<jwt>",  
  "code": "123456",  
  "client_id": "bigk_wallet"  
}
```

The code field accepts either a 6-digit TOTP code **or** a backup code. If the TOTP code doesn't match, the server tries matching against backup codes automatically. Each backup code can only be used once.

Response includes a X-KCC-ID-2FA-Verified: true header. The token set includes extra claims:

```
{  
  "access_token": "<jwt>",  
  "token_type": "Bearer",  
  "id_token": "<jwt>",  
  "refresh_token": "<uuid>",  
  "expires_in": 3600,  
}
```

```
  "scope": "openid profile email"
}
```

The issued access token contains "2fa_verified": true and "auth_methods": ["password", "totp"] claims.

Error (401 — invalid code):

```
{
  "error": "invalid_code",
  "error_description": "Invalid two-factor authentication code."
}
```

Error (401 — expired challenge):

```
{
  "error": "invalid_challenge",
  "error_description": "Session expired. Please sign in again."
}
```

6.4 Disable TOTP (requires Bearer token)

- **POST** /kccid/v1/2fa/totp/disable

```
{
  "password": "Secret123!"
}
```

Disabling TOTP also increments token_version, invalidating all existing sessions.

6.5 2FA Status (requires Bearer token)

- **GET** /kccid/v1/2fa/status

```
{
  "enabled": true,
  "enabled_at": "2026-05-15T10:30:00+00:00",
  "last_used_at": "2026-05-17T08:00:00+00:00",
  "remaining_backup_codes": 5
}
```

7) OTP & Registration

7.1 Request OTP

- **POST** /kccid/v1/otp/request (rate limited: throttle:otp)

```
{
  "channel": "email",
  "target": "user@example.com",
  "purpose": "registration",
  "client_id": "bigk_wallet"
}
```

7.2 Verify OTP

- **POST** /kccid/v1/otp/verify

```
{  
  "session_id": "<uuid>",  
  "code": "123456"  
}
```

7.3 Register

- **POST** /kccid/v1/register

```
{  
  "otp_session_id": "<uuid>",  
  "client_id": "bigk_wallet",  
  "display_name": "Jane Doe",  
  "handle": "jane_doe",  
  "password": "Secret123!",  
  "password_confirmation": "Secret123!"  
}
```

Password policy: min 8 chars, at least 1 uppercase, 1 digit, 1 special char from @\$!%*?&#.

8) Password Reset

- **POST** /kccid/v1/forgot-password — starts reset, sends OTP
- **POST** /kccid/v1/verify-reset-otp — verifies OTP, returns reset_token
- **POST** /kccid/v1/reset-password — sets new password using reset token

All active refresh tokens are revoked after a password reset.

9) Change Password (Authenticated)

- **POST** /kccid/v1/change-password

```
{  
  "current_password": "Secret123!",  
  "new_password": "NewSecret123!",  
  "new_password_confirmation": "NewSecret123!"  
}
```

10) Wallet Provisioning (Authenticated)

- **POST** /kccid/v1/provision-wallet

Provisions a wallet linkage for cross-ecosystem flows (e.g. Shuyi user getting a BigK wallet).

11) Admin Dashboard API

The KCC ID Dashboard (id.kccdigital.com) uses admin-specific endpoints.

11.1 Admin Login

- **POST** /kccid/v1/admin/login

11.2 Dashboard

Method	Endpoint	Description
GET	/kccid/v1/admin/dashboard/stats	User counts, login metrics
GET	/kccid/v1/admin/dashboard/login-trends	Login volume over time
GET	/kccid/v1/admin/dashboard/provider-distribution	Auth provider breakdown
GET	/kccid/v1/admin/dashboard/top-clients	Most active client applications

11.3 User Management

Method	Endpoint	Description
GET	/kccid/v1/admin/users	List users (paginated, searchable)
GET	/kccid/v1/admin/users/{userId}	Get user detail
PUT	/kccid/v1/admin/users/{userId}	Update user
POST	/kccid/v1/admin/users/{userId}/suspend	Suspend user
POST	/kccid/v1/admin/users/{userId}/activate	Activate user
POST	/kccid/v1/admin/users/{userId}/reset-password	Reset user's password
GET	/kccid/v1/admin/users/{userId}/sessions	User's active sessions
POST	/kccid/v1/admin/users/{userId}/sessions/revoke-all	Revoke all sessions
GET	/kccid/v1/admin/users/{userId}/audit-logs	User's audit trail

11.4 Audit Logs

Method	Endpoint	Description
GET	/kccid/v1/admin/audit-logs	All audit logs

Method	Endpoint	Description
GET	/kccid/v1/admin/audit-logs/actions	Available action types
GET	/kccid/v1/admin/audit-logs/export	Export logs as CSV

11.5 Client Management

Method	Endpoint	Description
GET	/kccid/v1/admin/clients	List registered clients
GET	/kccid/v1/admin/clients/{clientId}	Client detail
POST	/kccid/v1/admin/clients	Register new client

11.6 Security

Method	Endpoint	Description
GET	/kccid/v1/admin/security/metrics	Security overview
GET	/kccid/v1/admin/security/alerts	Active security alerts
POST	/kccid/v1/admin/security/alerts/{alertId}/resolve	Resolve alert
GET	/kccid/v1/admin/security/suspicious-ips	Suspicious IPs
POST	/kccid/v1/admin/security/block-ip	Block an IP
POST	/kccid/v1/admin/security/unblock-ip	Unblock an IP
GET	/kccid/v1/admin/security/blocked-ips	List blocked IPs

11.7 Settings

Method	Endpoint	Description
GET	/kccid/v1/admin/settings	Get service settings
PUT	/kccid/v1/admin/settings	Update settings

12) Token Format (JWT)

Standard claims

- iss — issuer (must equal KCC_ID_ISSUER)
- sub — KCC user ID (UUID)
- aud — client_id
- exp, iat
- scope

Authorization claims

- token_type (e.g. app, merchant, admin)
- role (e.g. app, merchant_admin, super_admin)
- wallet_id (optional)
- merchant_ids (optional)
- user_permissions (optional)
- token_version — integer, incremented when 2FA is enabled/disabled or password is changed
- 2fa_verified — boolean, present when login completed 2FA challenge (optional)
- auth_methods — array of auth methods used, e.g. ["password", "totp"] (optional)

Token Version

token_version is a monotonically increasing integer stored on the kcc_users and kcc_refresh_tokens tables. Every access token, ID token, challenge token, and refresh token includes the user's current token_version at issuance.

Validation: The kcc.token middleware and the 2FA verify endpoint both check that the token's token_version matches the user's current token_version. If they don't match, the token is rejected with 401.

When it increments:

- Enabling TOTP (/2fa/totp/enable)
- Disabling TOTP (/2fa/totp/disable)
- Password change (/change-password)
- Password reset (/reset-password)

This mechanism immediately invalidates all existing tokens and sessions without requiring a token revocation sweep.

Sample decoded access token

```
{  
  "iss": "https://auth.bigkpay.com",  
  "sub": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaaa",  
  "aud": "shuyi",  
  "exp": 1760000000,  
}
```

```
{
  "iat": 1759996400,
  "scope": "openid profile email",
  "token_type": "app",
  "role": "app",
  "wallet_id": 123,
  "token_version": 2
}
```

13) Internal API (Service-to-Service)

13.1 Authorization sync

- **POST** /internal/authorizations/sync
- Header: X-KCC-ID-SYNC-SECRET: <shared_secret>

13.2 Ensure user

- **POST** /internal/users/ensure
 - Header: X-KCC-ID-SYNC-SECRET: <shared_secret>
 - Only accepts password_hash (plain-text password returns 422)
-

14) Error Handling

Code	Meaning
200	Success
201	Resource created
400	Validation / verification failure
401	Unauthorized / invalid token
403	Access denied
409	Conflict (account already exists)
422	Validation error
429	Rate limit exceeded

Rate Limits

Throttle key	Default limit	Applied to
throttle:login	5 / min per IP	/authorize, /login
throttle:otp	5 / hour per IP	/otp/request, 2fa/*

14) Recommended Integration Flows

14.1 Web/Mobile (recommended): PKCE

1. Collect identifier + password + generate PKCE

2. POST /kccid/v1/authorize → returns code
3. POST /kccid/v1/token with grant_type=authorization_code → returns tokens
4. Use access_token against downstream services in the ecosystem
5. Refresh with grant_type=refresh_token via POST /kccid/v1/token when needed

14.2 Registration

1. POST /kccid/v1/otp/request (purpose registration)
2. POST /kccid/v1/otp/verify
3. POST /kccid/v1/register
4. Login via PKCE or /kccid/v1/login

14.3 “Shuyi or Kincore user provisions BigK wallet” (high-level)

1. User registers/logs in on Shuyi or Kincore using KCC ID
2. Shuyi or Kincore obtains access token (aud=shuyi or aud=kincore)
3. Shuyi or Kincore calls POST /kccid/v1/provision-wallet
4. Shuyi or Kincore uses the resulting wallet linkage + downstream BigK services (depending on your ecosystem’s chosen linking strategy)

14.4 Multi-app / multi-client behavior

KCC ID is designed around:

- **one central identity** per user (kcc_user_id)
- **multiple client-specific authorization records** for that same identity
- **client-scoped tokens** issued for a specific client_id

Practical implications:

- a user may have one identity and multiple active authorizations, such as bigk_wallet, bigk_merchant, and shuyi
- tokens are issued in the context of a requested client_id
- downstream services should validate aud and should not assume that a token issued for one client is automatically appropriate for another client

cross-app SSO in the ecosystem should be understood as shared identity with per-client authorization, not as one unrestricted token reused across all products

Document owner: Munene